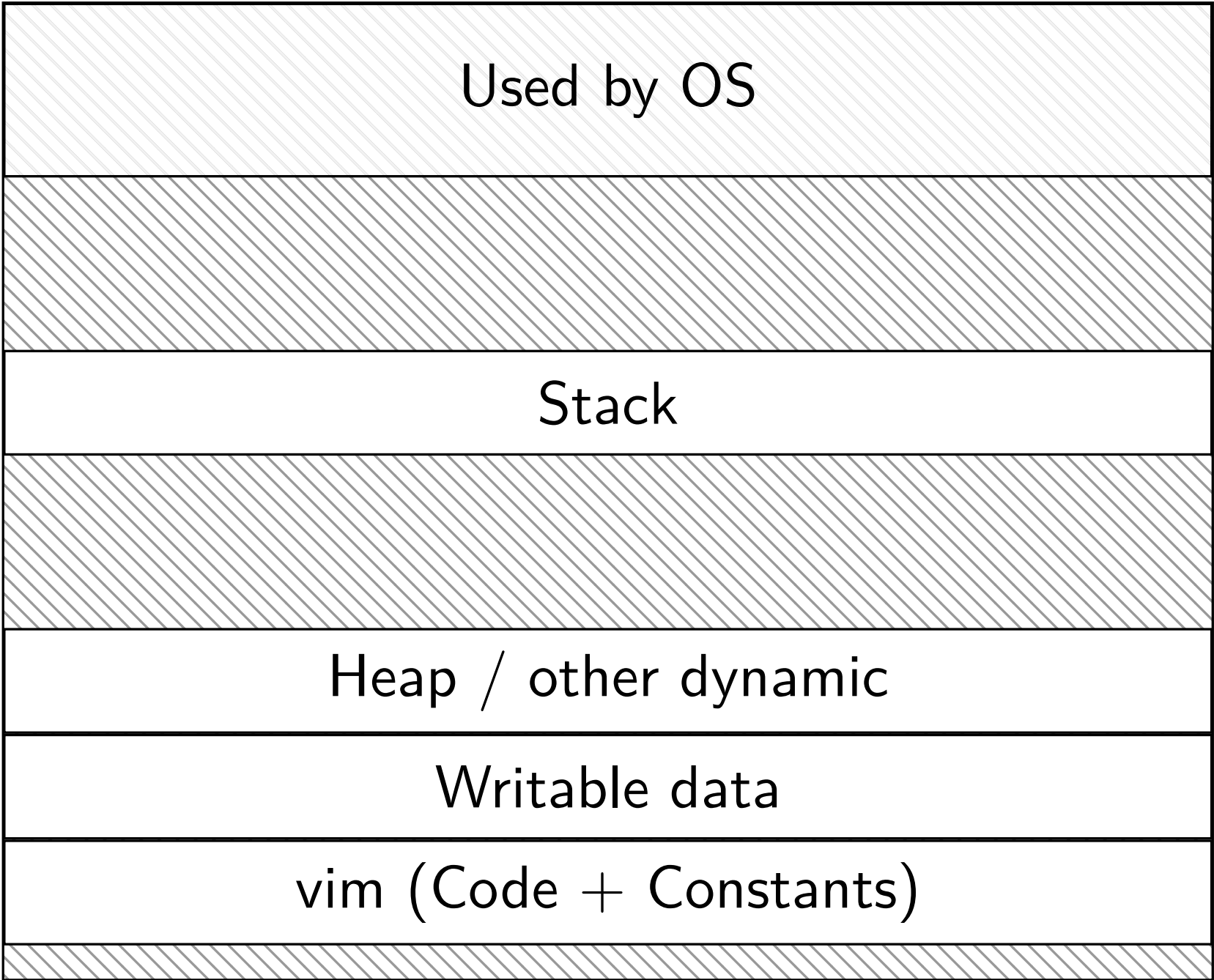
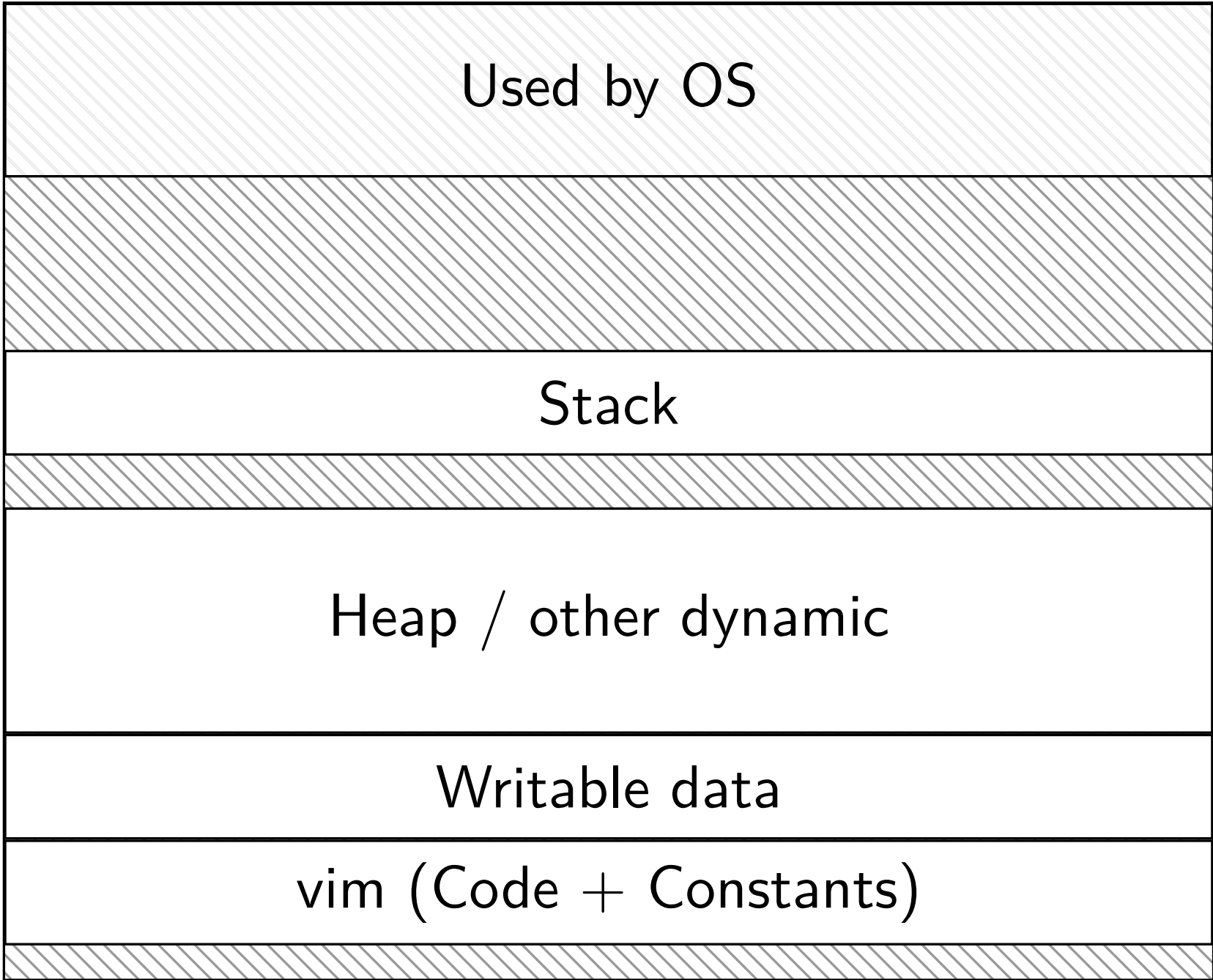


vim (two copies)

Vim (run by user mst3k)

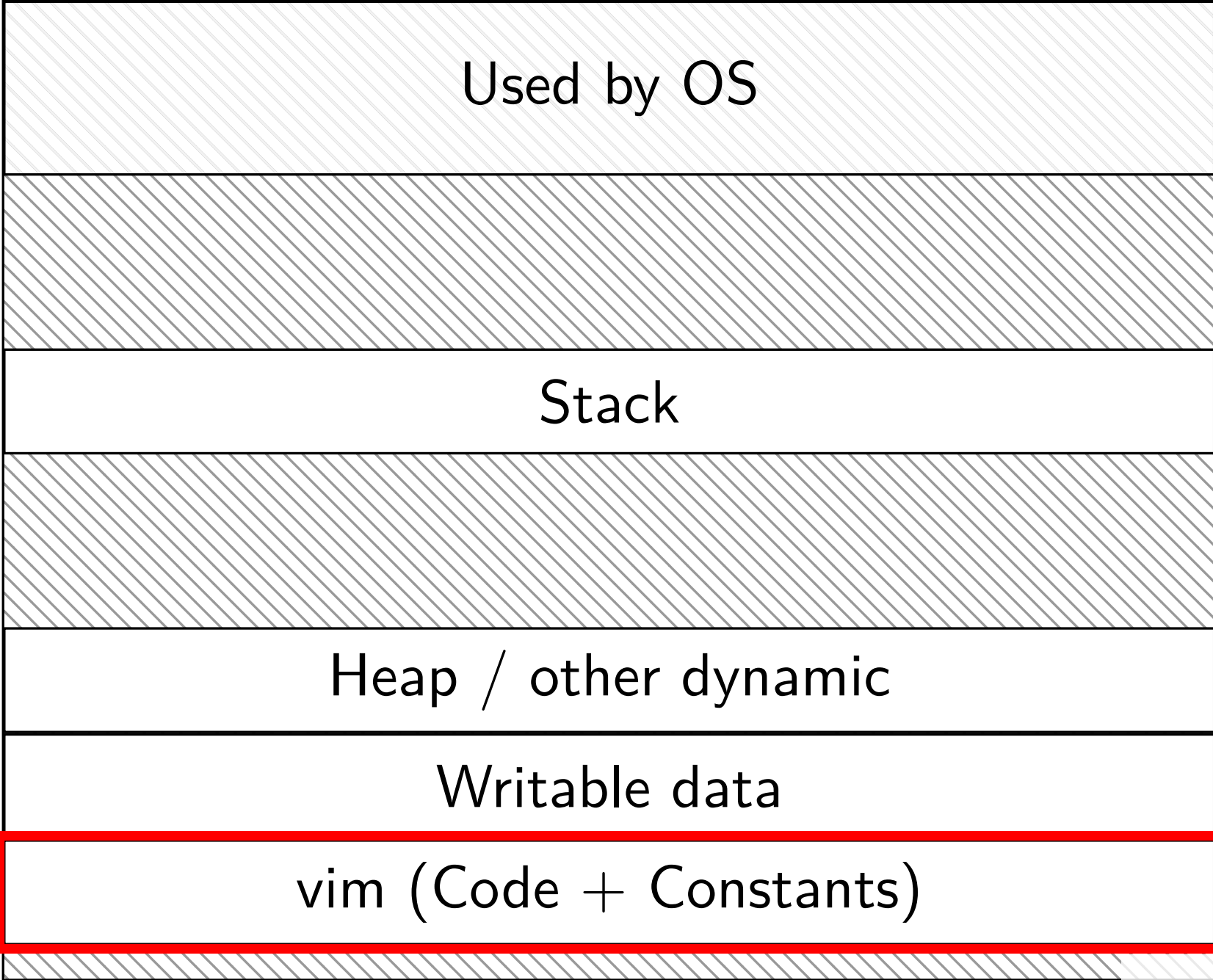


Vim (run by user xyz4w)

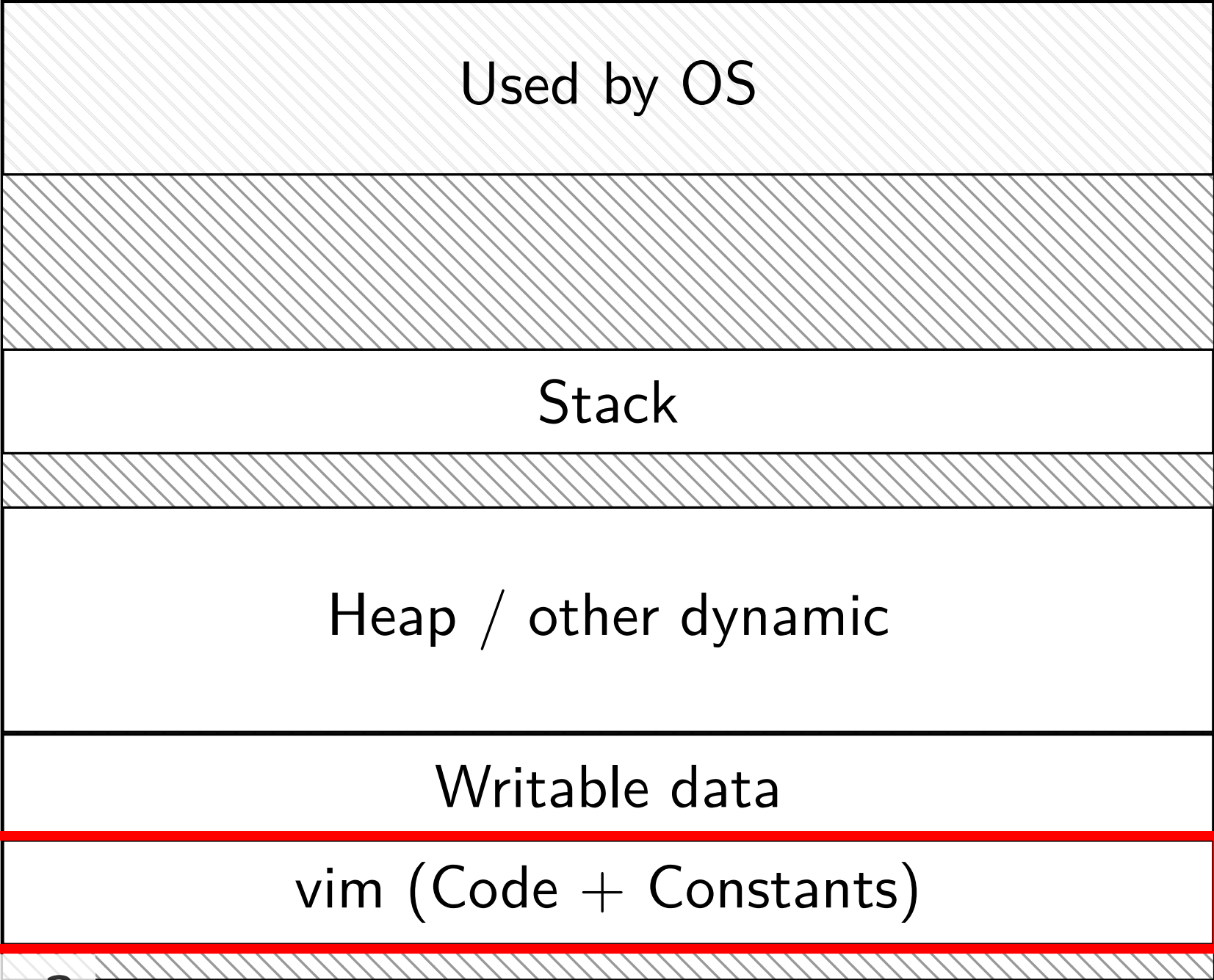


vim (two copies)

Vim (run by user mst3k)



Vim (run by user xyz4w)



same data?

two copies of program

would like to only have one copy of program code

could setup both page tables to point to same code, but...

what if `mst3k`'s vim has bug and modifies its code

expect this to break `mst3k`'s vim, but...

would also break `xyz4w`'s vim

violates process abstraction:

“illusion of own memory”

permissions bits

page table entry will have more *permissions bits*

- can access in user mode?
- can read from?
- can write to?
- can execute from?

checked by hardware like valid bit

page table (logically)

virtual page #	valid?	user?	write?	exec?	physical page #
0000 0000	0	0	0	0	00 0000 0000
0000 0001	1	1	1	0	10 0010 0110
0000 0010	1	1	1	0	00 0000 1100
0000 0011	1	1	0	1	11 0000 0011
...					
1111 1111	1	0	1	0	00 1110 1000

running OS code

system calls, I/O events, etc. run OS code in kernel mode

where in memory is this OS code?

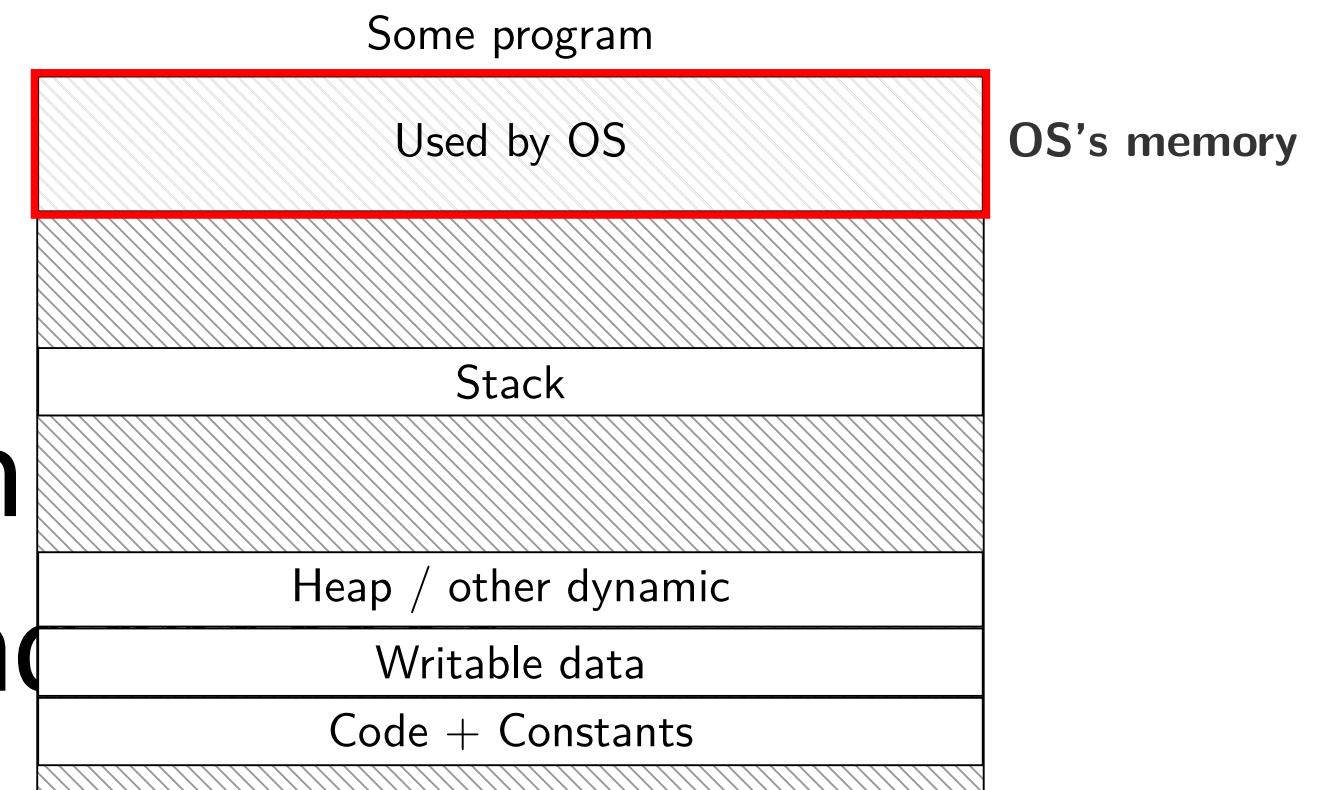
- probably have a page table entry pointing to it
- marked not accessible in user mode

code better not be modified by user program

- otherwise: uncontrolled way to “escape” user mode

OS memory in programs

most OSes: kernel runs using
normal programs' page tables:
avoids page table change on exception
⇒ kernel code/data needs virtual address



typically: high-numbered virtual pages reserved for this
those page table entries marked non-user-accessible
(can't allow programs to mess with OS's stuff)
usually copied in every process's page table

invalid pages

model so far: page tables = logical memory of program

invalid page $\implies$ process accessing it should crash

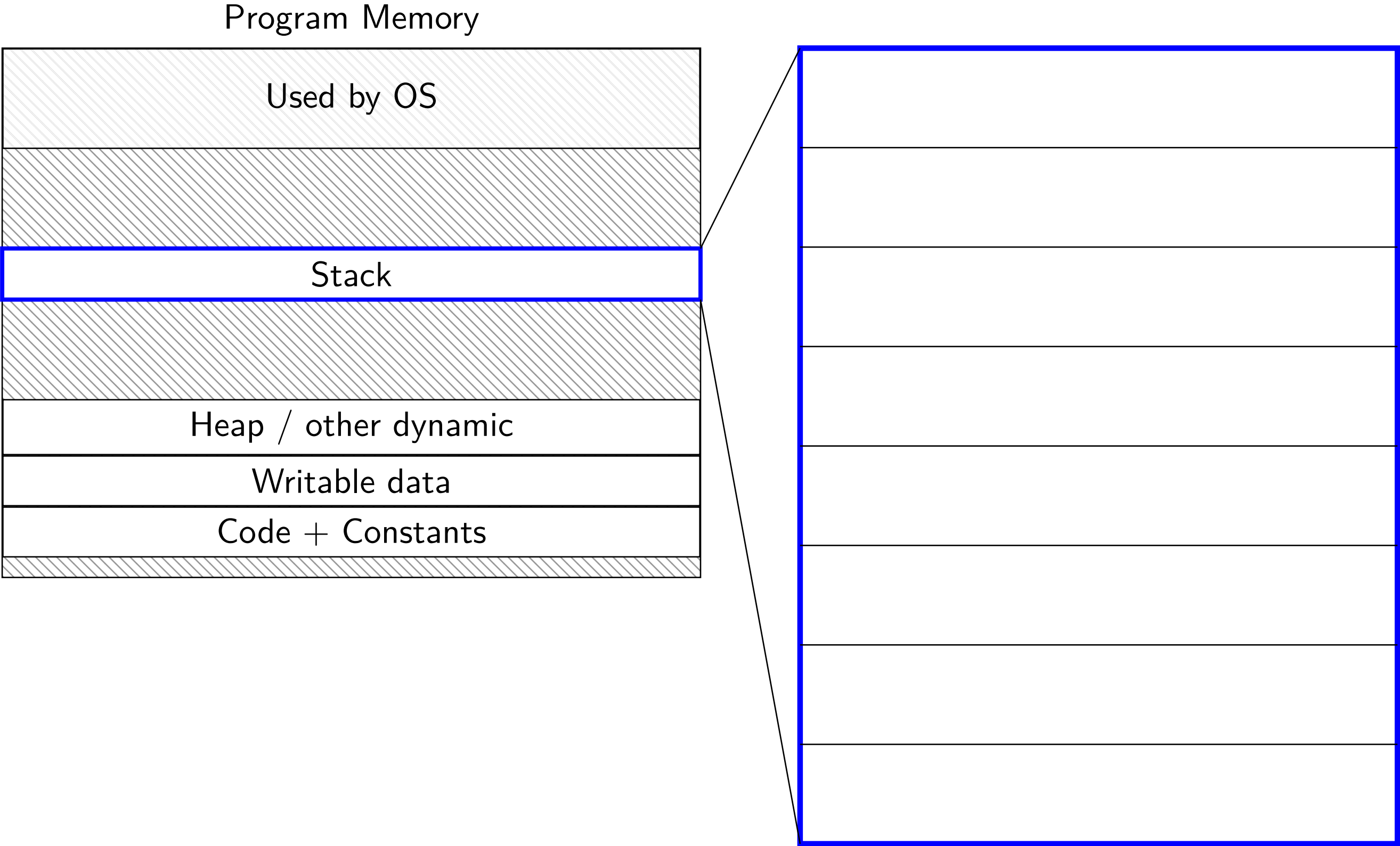
wrong permissions $\implies$ process accessing it should crash

but it doesn't need to work that way

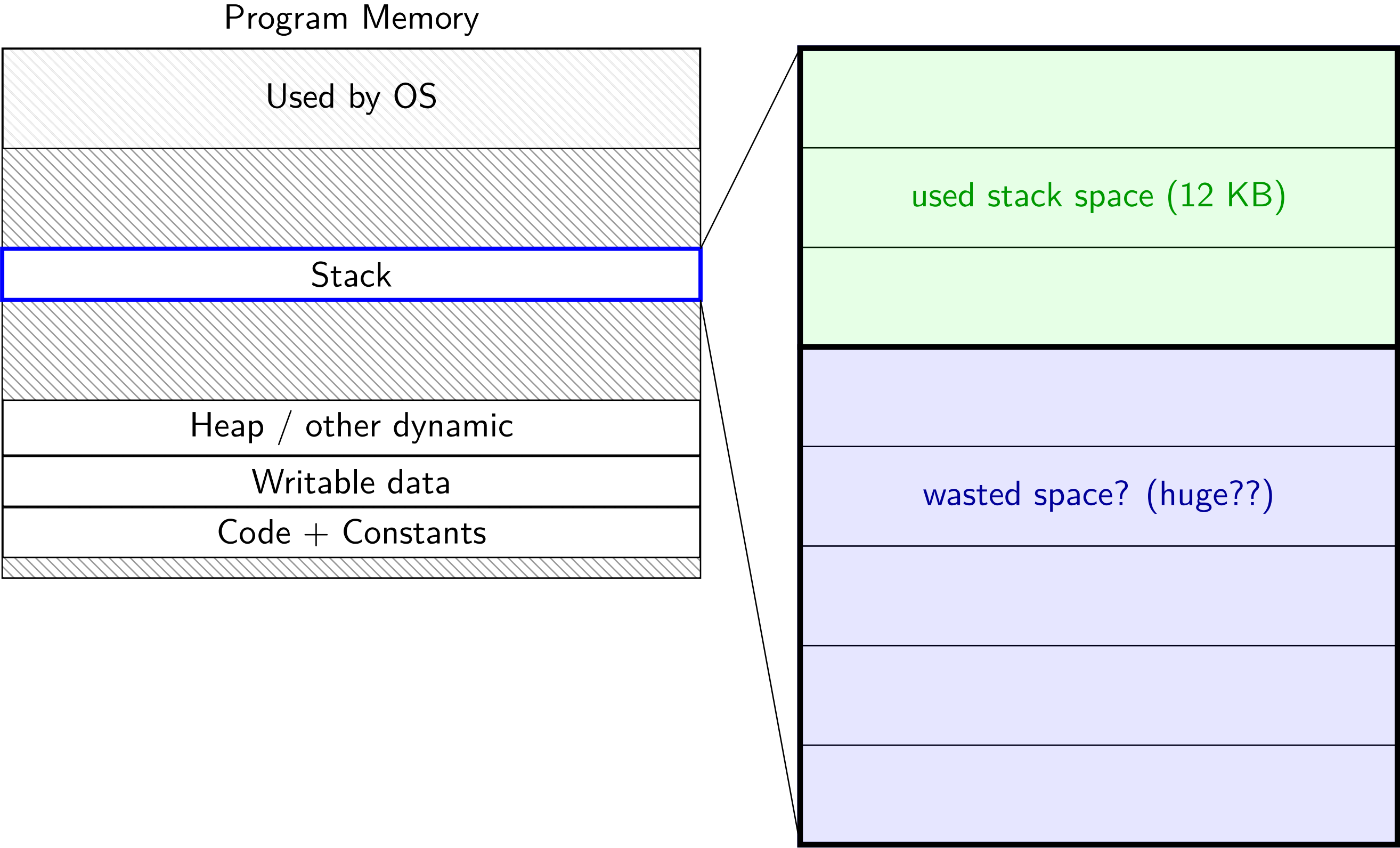
and doesn't work that way in most OSes

gives operating system more flexibility!

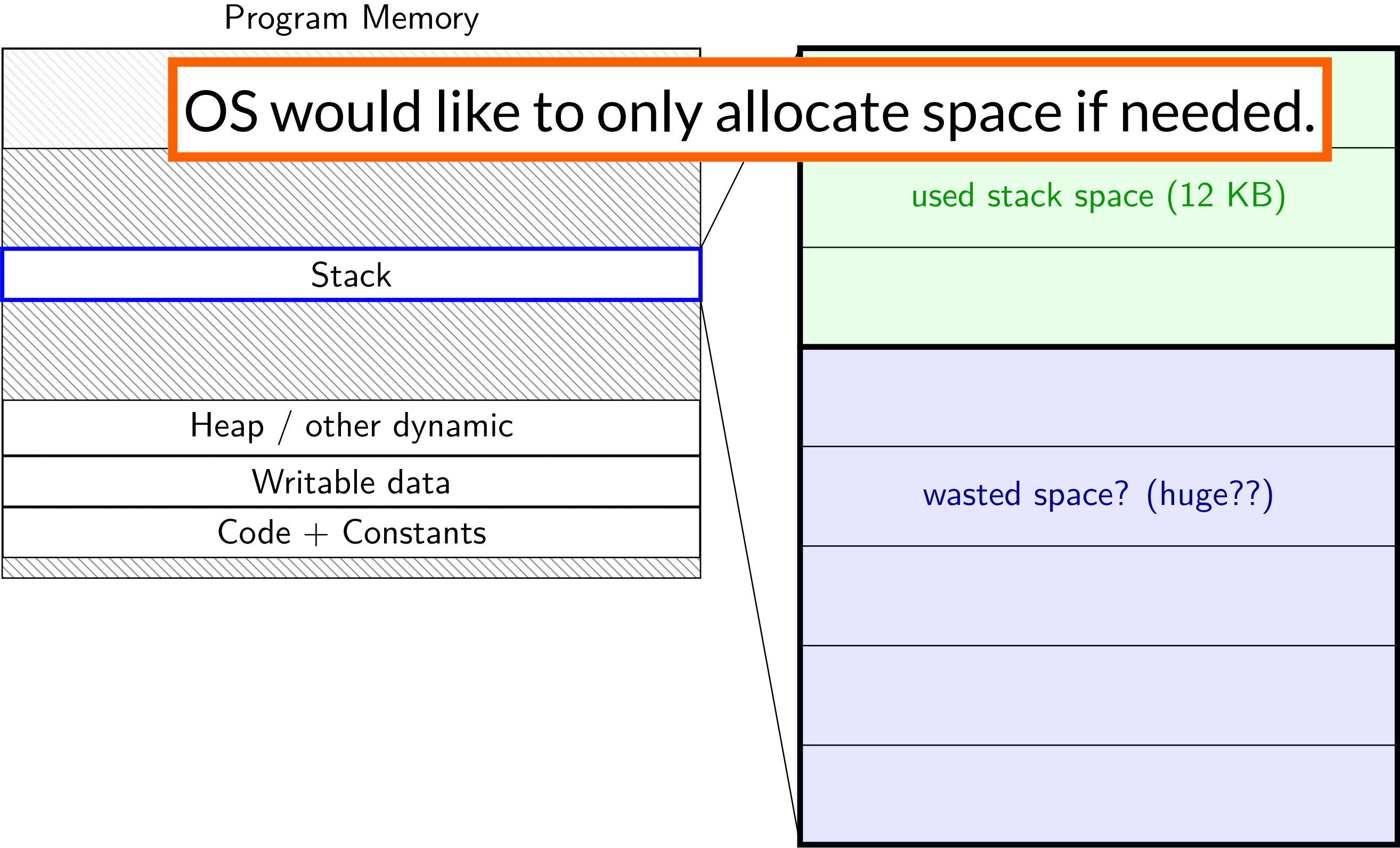
space on demand



space on demand



space on demand



allocating space on demand

`%rsp = 0x7FFFC000`

```
...  
// requires more stack space  
A: pushq %rbx  
B: movq 8(%rcx), %rbx  
C: addq %rbx, %rax  
...
```

VPN
...
0x7FFFB
0x7FFFC
0x7FFFD
0x7FFFE
0x7FFFF
...

valid?	physical page
...	...
0	---
1	0x200DF
1	0x12340
1	0x12347
1	0x12345
...	...

allocating space on demand

`%rsp = 0x7FFFC000`

```
...  
// requires more stack space  
A: pushq %rbx → fault!  
B: movq 8(%rcx), %rbx  
C: addq %rbx, %rax  
...
```

VPN
...
0x7FFFB
0x7FFFC
0x7FFFD
0x7FFFE
0x7FFFF
...

valid?	physical page
...	...
0	---
1	0x200DF
1	0x12340
1	0x12347
1	0x12345
...	...

pushq triggers exception
hardware says “accessing address 0x7FFBFF8”
OS looks up what’s should be there — “stack”

allocating space on demand

`%rsp = 0x7FFFC000`

```
...  
// requires more stack space  
A: pushq %rbx restarted  
B: movq 8(%rcx), %rbx  
C: addq %rbx, %rax  
...
```

VPN

...
0x7FFFB
0x7FFFC
0x7FFFD
0x7FFFE
0x7FFFF
...

valid? physical page

...	...
1	0x200D8
1	0x200DF
1	0x12340
1	0x12347
1	0x12345
...	...

in exception handler, OS allocates more stack space
OS updates the page table
then returns to retry the instruction

allocating space on demand

note: the space doesn't have to be initially empty
load from file, etc. instead of allocating empty page

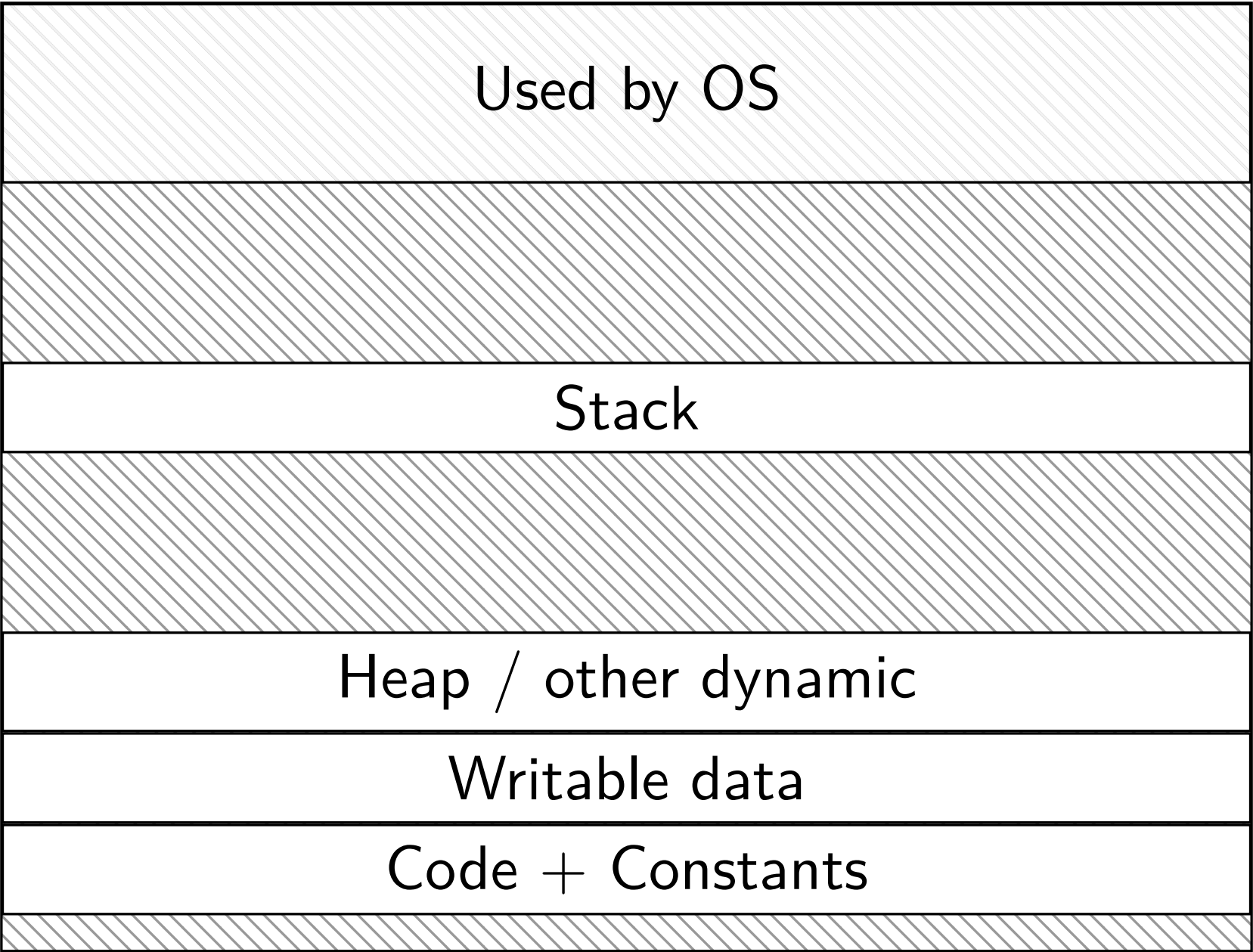
loading program can be *merely creating empty page table*

everything else can be handled *in response to page faults*

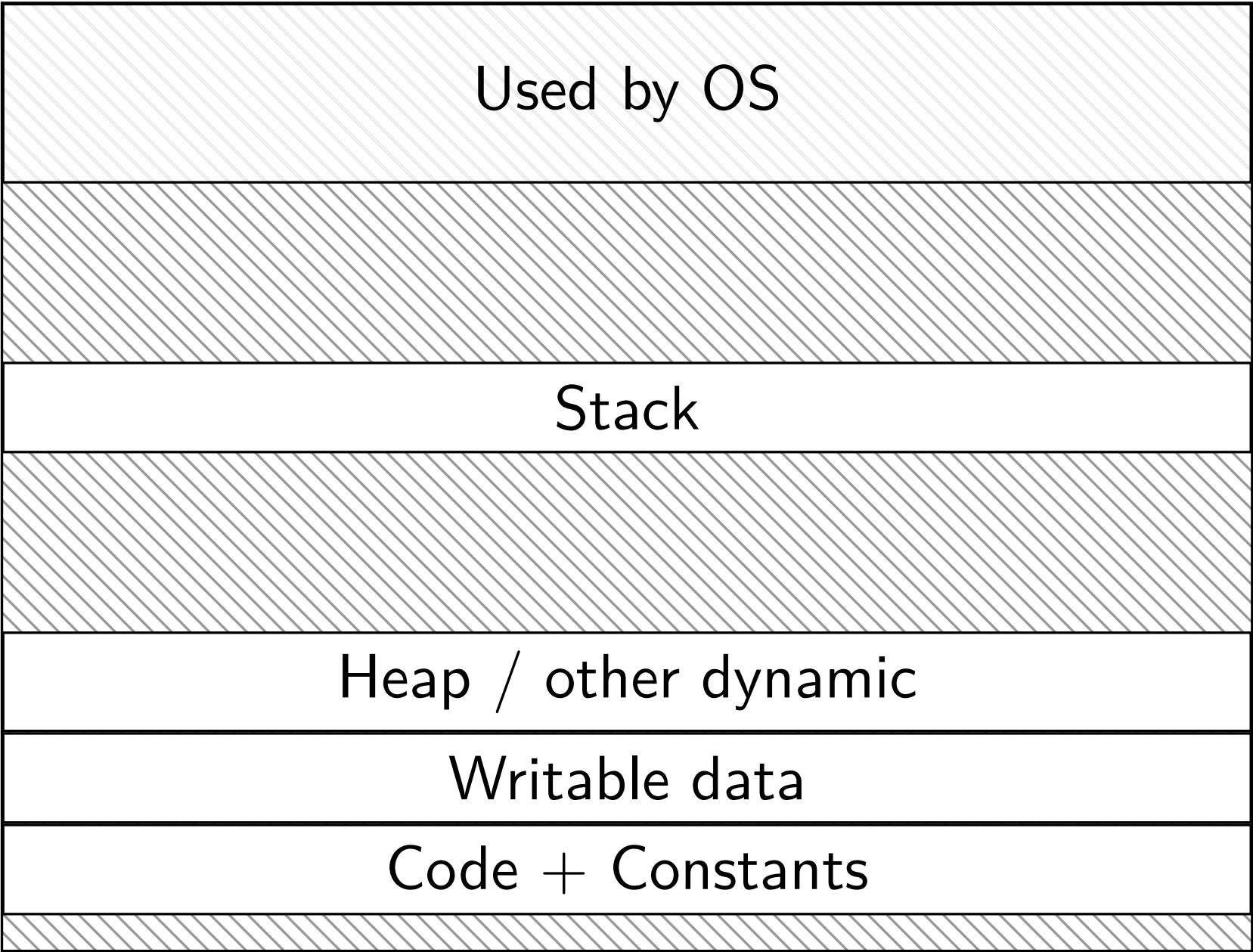
page fault = exception triggered for invalid/unpermissions page
no time/space spent loading/allocating unneeded space

do we really need a complete copy?

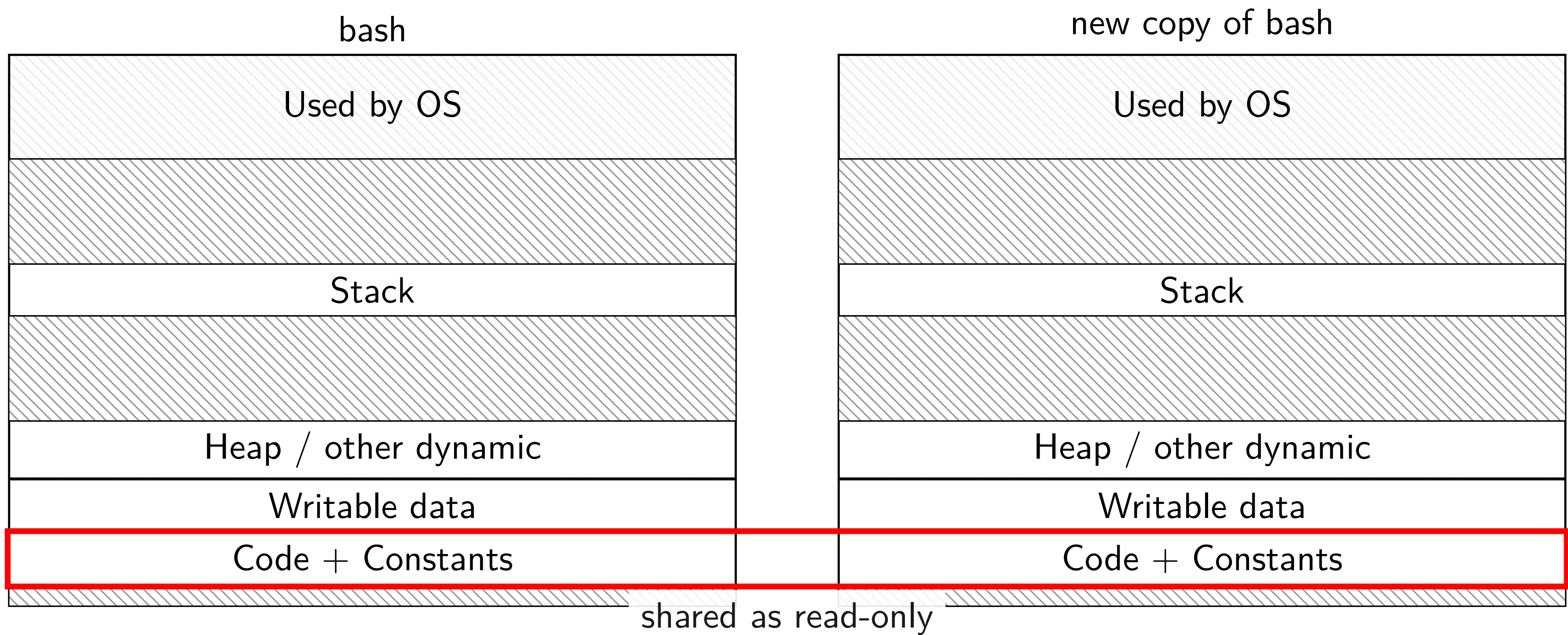
bash



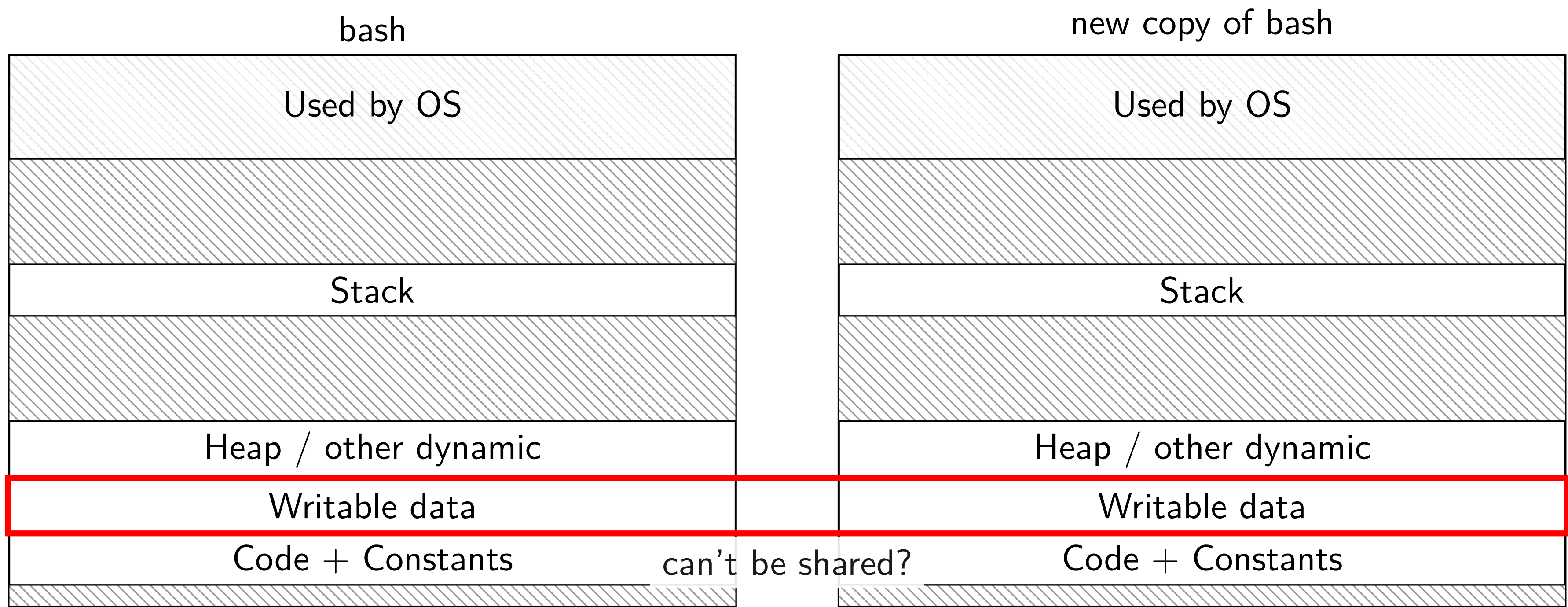
new copy of bash



do we really need a complete copy?



do we really need a complete copy?



trick for extra sharing

sharing writeable data is fine — until either process modifies it

- example: default value of global variables

- might typically not change

- (or OS might have preloaded executable's data anyways)

can we detect modifications?

trick: tell CPU (via page table) shared part is read-only

processor will trigger a fault when it's written

copy-on-write and page tables

VPN	valid?	write?	physical page
...	...	...	...
0x00601	1	1	0x12345
0x00602	1	1	0x12347
0x00603	1	1	0x12340
0x00604	1	1	0x200DF
0x00605	1	1	0x200AF
...	...	...	...

copy-on-write and page tables

VPN	valid?	write?	physical page
...	...	...	...
0x00601	1	0	0x12345
0x00602	1	0	0x12347
0x00603	1	0	0x12340
0x00604	1	0	0x200DF
0x00605	1	0	0x200AF
...	...	...	...

VPN	valid?	write?	physical page
...	...	...	...
0x00601	1	0	0x12345
0x00602	1	0	0x12347
0x00603	1	0	0x12340
0x00604	1	0	0x200DF
0x00605	1	0	0x200AF
...	...	...	...

copy operation actually duplicates page table
both processes *share all physical pages*
but marked *read-only in both tables*

copy-on-write and page tables

VPN	valid?	write?	physical page
...	...	...	...
0x00601	1	0	0x12345
0x00602	1	0	0x12347
0x00603	1	0	0x12340
0x00604	1	0	0x200DF
0x00605	1	0	0x200AF
...	...	...	...

VPN	valid?	write?	physical page
...	...	...	...
0x00601	1	0	0x12345
0x00602	1	0	0x12347
0x00603	1	0	0x12340
0x00604	1	0	0x200DF
0x00605	1	0	0x200AF
...	...	...	...

when either process tries to write a read-only page
triggers a page fault – OS actually copies the page

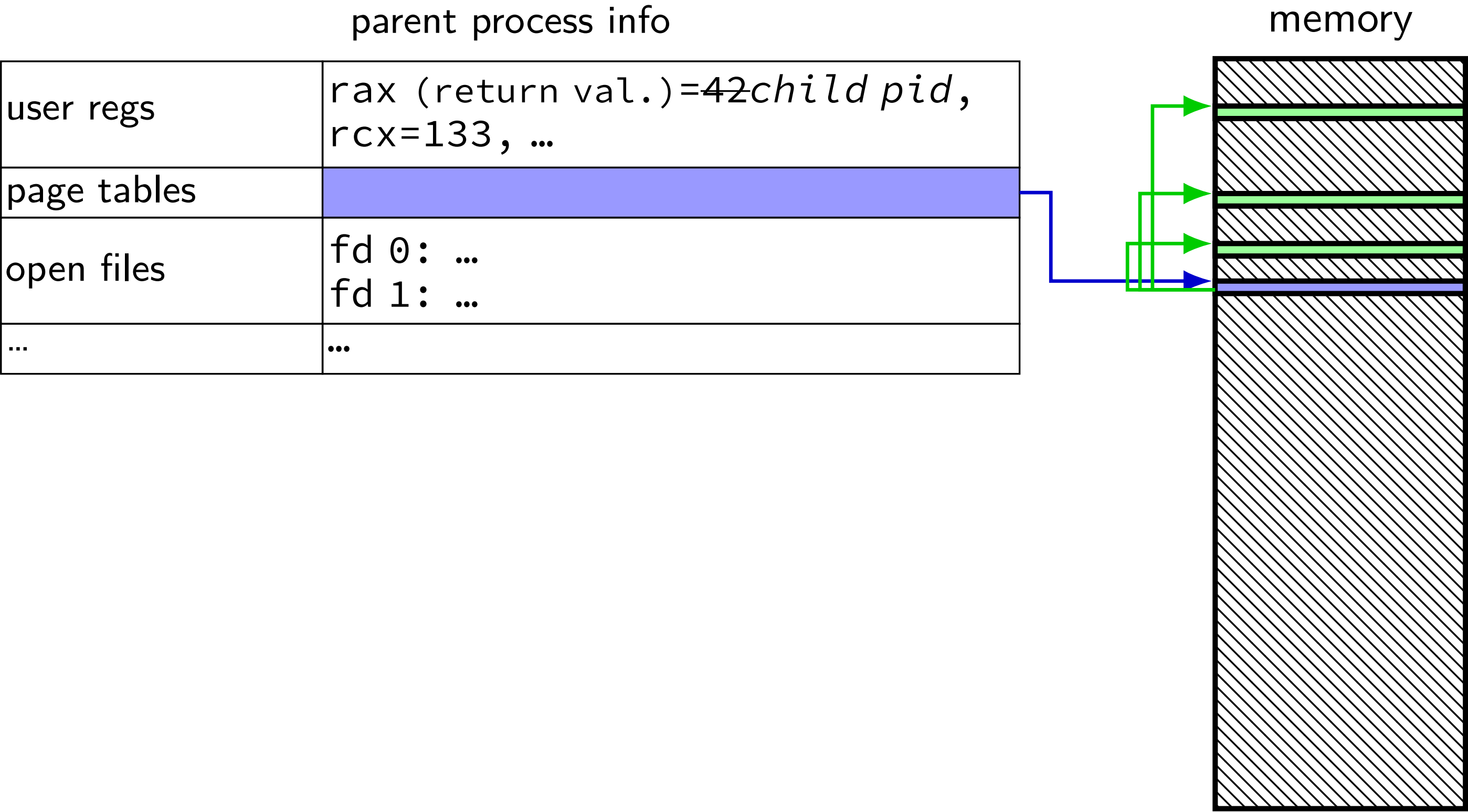
copy-on-write and page tables

VPN	valid?	write?	physical page
...	...	...	...
0x00601	1	0	0x12345
0x00602	1	0	0x12347
0x00603	1	0	0x12340
0x00604	1	0	0x200DF
0x00605	1	0	0x200AF
...	...	...	...

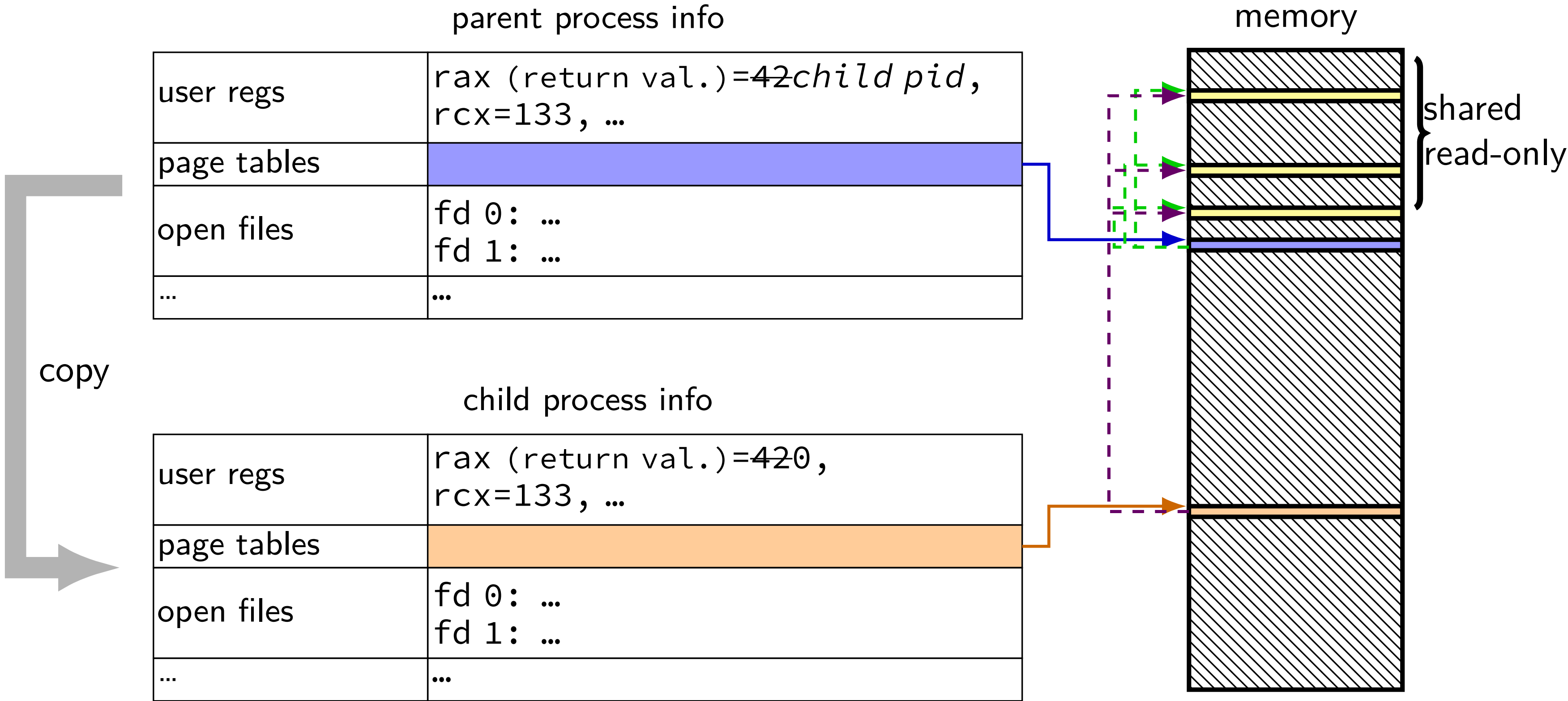
VPN	valid?	write?	physical page
...	...	...	...
0x00601	1	0	0x12345
0x00602	1	0	0x12347
0x00603	1	0	0x12340
0x00604	1	0	0x200DF
0x00605	1	1	0x300FD
...	...	...	...

after allocating a copy, OS reruns the write instruction

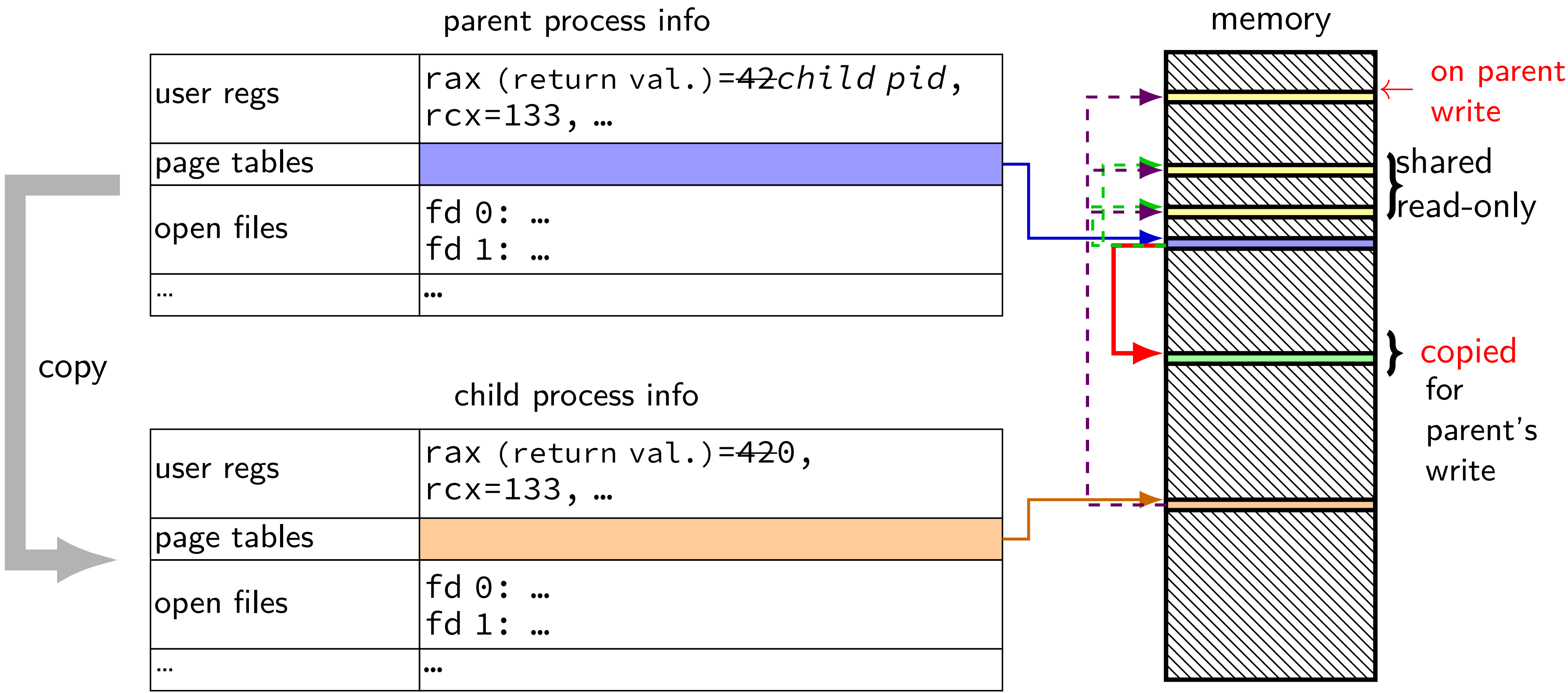
fork (w/ copy-on-write, if parent writes first)



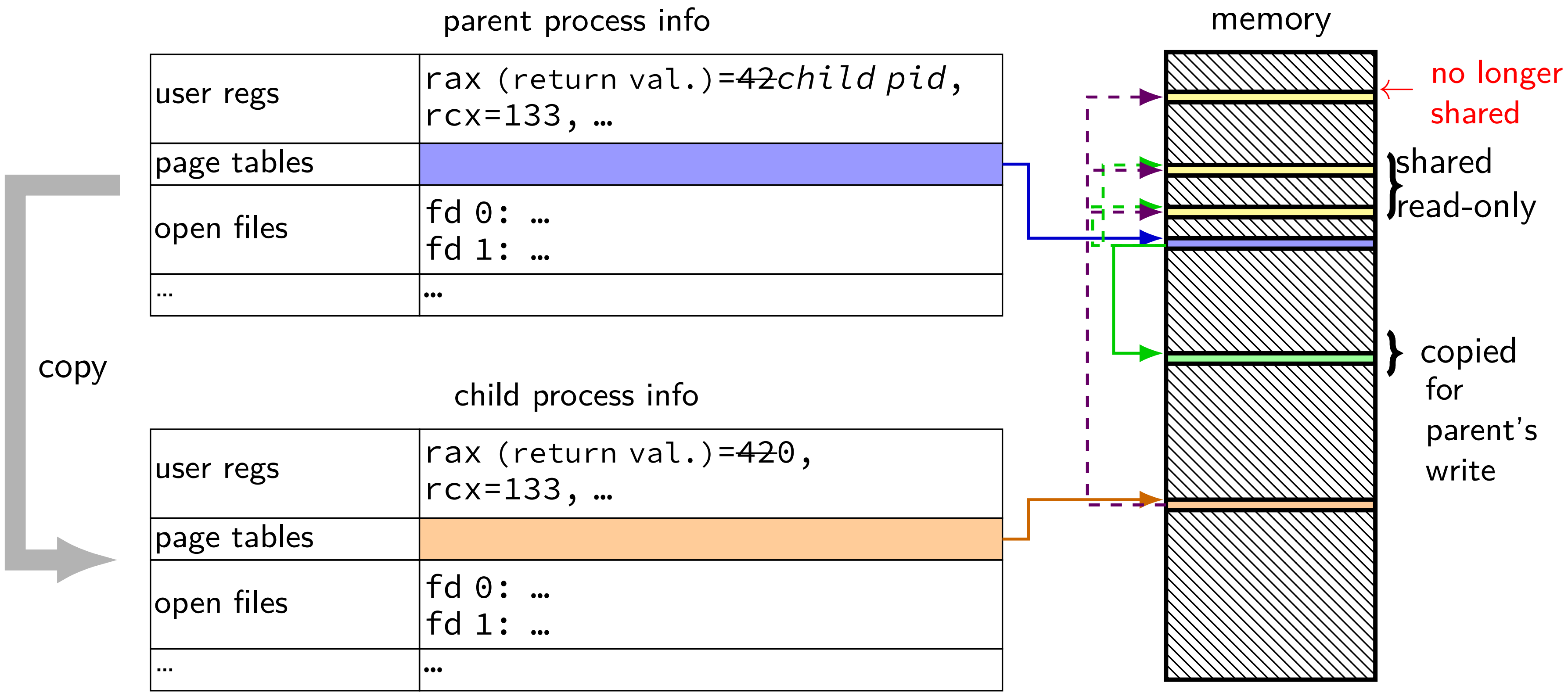
fork (w/ copy-on-write, if parent writes first)



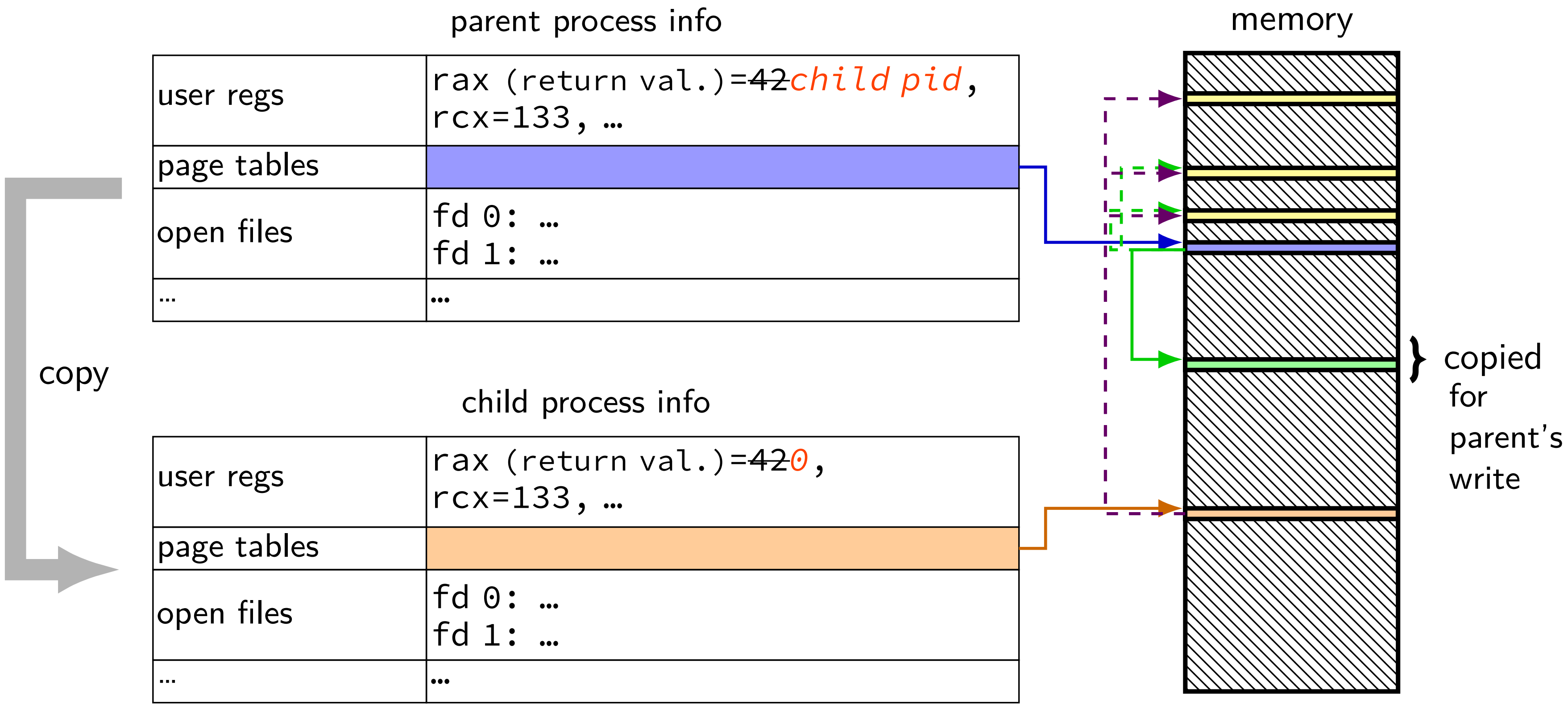
fork (w/ copy-on-write, if parent writes first)



fork (w/ copy-on-write, if parent writes first)



fork (w/ copy-on-write, if parent writes first)



mmap

Linux/Unix has a function to “map” a file to memory

```
int file = open("somefile.dat", O_RDWR);

// data is region of memory that represents file
char *data = mmap(..., file, 0);

// read byte 6 from somefile.dat
char seventh_char = data[6];

// modifies byte 100 of somefile.dat
data[100] = 'x';
// can continue to use 'data' like an array
```

Linux memory tracking

Linux kernel treats all program memory as special case of mmap'd file

two major features to help

- ‘private’ mappings = copy-on-write from original file

 - supports ‘load program from disk on demand’

 - pages not shared with original file after writes

- original file = all zeroes (instead of file from disk)

 - “anonymous mapping”

 - handles ‘allocate-on-demand’ scenario

Linux maps: list of maps

```
$ cat /proc/self/maps
00400000-0040b000 r-xp 00000000 08:01 48328831      /bin/cat
0060a000-0060b000 r--p 0000a000 08:01 48328831      /bin/cat
0060b000-0060c000 rw-p 0000b000 08:01 48328831      /bin/cat
01974000-01995000 rw-p 00000000 00:00 0          [heap]
7f60c718b000-7f60c7490000 r--p 00000000 08:01 77483660 /usr/lib/locale/locale-archive
7f60c7490000-7f60c764e000 r-xp 00000000 08:01 96659129 /lib/x86_64-linux-gnu/libc-2.19.so
7f60c764e000-7f60c784e000 ---p 001be000 08:01 96659129 /lib/x86_64-linux-gnu/libc-2.19.so
7f60c784e000-7f60c7852000 r--p 001be000 08:01 96659129 /lib/x86_64-linux-gnu/libc-2.19.so
7f60c7852000-7f60c7854000 rw-p 001c2000 08:01 96659129 /lib/x86_64-linux-gnu/libc-2.19.so
7f60c7854000-7f60c7859000 rw-p 00000000 00:00 0
7f60c7859000-7f60c787c000 r-xp 00000000 08:01 96659109 /lib/x86_64-linux-gnu/ld-2.19.so
7f60c7a39000-7f60c7a3b000 rw-p 00000000 00:00 0
7f60c7a7a000-7f60c7a7b000 rw-p 00000000 00:00 0
7f60c7a7b000-7f60c7a7c000 r--p 00022000 08:01 96659109 /lib/x86_64-linux-gnu/ld-2.19.so
7f60c7a7c000-7f60c7a7d000 rw-p 00023000 08:01 96659109 /lib/x86_64-linux-gnu/ld-2.19.so
7f60c7a7d000-7f60c7a7e000 rw-p 00000000 00:00 0
7ffc5d2b2000-7ffc5d2d3000 rw-p 00000000 00:00 0          [stack]
7ffc5d3b0000-7ffc5d3b3000 r--p 00000000 00:00 0          [vvar]
7ffc5d3b3000-7ffc5d3b5000 r-xp 00000000 00:00 0          [vdso]
fffffffffff600000-fffffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
```

Linux maps: list of maps

```
$ cat /proc/self/maps
00400000-0040b000 r-xp 00000000 08:01 48328831 /bin/cat
0060a000-0060b000 r--p 0000a000 08:01 48328831 /bin/cat
0060b000-0060c000 rw-p 0000b000 08:01 48328831 /bin/cat
01974000-01995000 rw-p 00000000 00:00 0 [heap]
7f60c718b000-7f60c7490000 r--p 00000000 08:01 77483660 /usr/lib/locale/locale-archive
7f60c7490000-7f60c764e000 r-xp 00000000 08:01 96659129 /lib/x86_64-linux-gnu/libc-2.19.so
7f60c764e000-7f60c784e000 ---p 001be000 08:01 96659129 /lib/x86_64-linux-gnu/libc-2.19.so
7f60c784e000-7f60c7852000 r--p 001be000 08:01 96659129 /lib/x86_64-linux-gnu/libc-2.19.so
7f60c7852000-7f60c7854000 rw-p 001c2000 08:01 96659129 /lib/x86_64-linux-gnu/libc-2.19.so
7f60c7854000-7f60c7859000 rw-p 00000000 00:00 0
7f60c7859000-7f60c787c000 r-xp 00000000 08:01 96659109 /lib/x86_64-linux-gnu/ld-2.19.so
7f60c7a39000-7f60c7a3b000 rw-p 00000000 00:00 0
7f60c7a7a000-7f60c7a7b000 rw-p 00000000 00:00 0
7f60c7a7b000-7f60c7a7c000 r--p 00022000 08:01 96659109 /lib/x86_64-linux-gnu/ld-2.19.so
7f60c7a7c000-7f60c7a7d000 rw-p 00023000 08:01 96659109 /lib/x86_64-linux-gnu/ld-2.19.so
7f60c7a7d000-7f60c7a7e000 rw-p 00000000 00:00 0
7ffc5d2b2000-7ffc5d2d3000 rw-p 00000000 00:00 0 [stack]
7ffc5d3b0000-7ffc5d3b3000 r--p 00000000 00:00 0 [vvar]
7ffc5d3b3000-7ffc5d3b5000 r-xp 00000000 00:00 0 [vdso]
ffffffffffffff600000-ffffffffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
```

Linux maps: list of maps

```
$ cat /proc/self/maps
00400000-0040b000 r-xp 00000000 08:01 48328831 /bin/cat
0060a000-0060b000 r--p 0000a000 08:01 48328831 /bin/cat
0060b000-0060c000 rw-p 0000b000 08:01 48328831 /bin/cat
01974000-01995000 rw-p 00000000 00:00 0 [heap]
7f60c718b000-7f60c7490000 r--p 00000000 00:00 0 [anon]
7f60c7490000-7f60c764e000 r-xp 00000000 00:00 0 [anon]
7f60c764e000-7f60c784e000 ---p 00100000 00:00 0 [anon]
7f60c784e000-7f60c7852000 r--p 00100000 00:00 0 [anon]
7f60c7852000-7f60c7854000 rw-p 00100000 00:00 0 [anon]
7f60c7854000-7f60c7859000 rw-p 00000000 00:00 0 [anon]
7f60c7859000-7f60c787c000 r-xp 00000000 00:00 0 [anon]
7f60c7a39000-7f60c7a3b000 rw-p 00000000 00:00 0 [anon]
7f60c7a7a000-7f60c7a7b000 rw-p 00000000 00:00 0 [anon]
7f60c7a7b000-7f60c7a7c000 r--p 00022000 08:01 96659109 /lib/x86_64-linux-gnu/ld-2.19.so
7f60c7a7c000-7f60c7a7d000 rw-p 00023000 08:01 96659109 /lib/x86_64-linux-gnu/ld-2.19.so
7f60c7a7d000-7f60c7a7e000 rw-p 00000000 00:00 0 [anon]
7ffc5d2b2000-7ffc5d2d3000 rw-p 00000000 00:00 0 [stack]
7ffc5d3b0000-7ffc5d3b3000 r--p 00000000 00:00 0 [vvar]
7ffc5d3b3000-7ffc5d3b5000 r-xp 00000000 00:00 0 [vdso]
fffffffffff600000-fffffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
```

OS tracks list of struct vm_area_struct with:
virtual address start, end
permissions
offset in backing file (if any)
pointer to backing file (if any)
(not shown):
info about sharing of non-file data

page tricks generally

deliberately *make program trigger page/protection fault*

but *don't assume page/protection fault is an error*

have *seperate data structures* represent logically allocated memory

e.g. “addresses 0x7FFF8000 to 0x7FFFFFFF are the stack”

page table is for the hardware and not the OS

example page table tricks

allocating space on demand

loading code/data from files on disk on demand

copy-on-write

saving data temporarily to disk, reloading to memory on demand
“swapping”

detecting whether memory was read/written recently

stopping in a debugger when a variable is modified

sharing memory between programs on two different machines

example page table tricks

allocating space on demand

loading code/data from files on disk on demand

copy-on-write

saving data temporarily to disk, reloading to memory on demand

“swapping”

detecting whether memory was read/written recently

stopping in a debugger when a variable is modified

sharing memory between programs on two different machines

example page table tricks

allocating space on demand

loading code/data from files on disk on demand

copy-on-write

saving data temporarily to disk, reloading to memory on demand
“swapping”

detecting whether memory was read/written recently

stopping in a debugger when a variable is modified

sharing memory between programs on two different machines

example page table tricks

allocating space on demand

loading code/data from files on disk on demand

copy-on-write

saving data temporarily to disk, reloading to memory on demand
“swapping”

detecting whether memory was read/written recently

stopping in a debugger when a variable is modified

sharing memory between programs on two different machines

example page table tricks

allocating space on demand

loading code/data from files on disk on demand

copy-on-write

saving data temporarily to disk, reloading to memory on demand
“swapping”

detecting whether memory was read/written recently

stopping in a debugger when a variable is modified

sharing memory between programs on two different machines

hardware help for page table tricks

information about the address causing the fault

- e.g. special register with memory address accessed

- harder alternative: OS disassembles instruction, look at registers

(by default) rerun faulting instruction when returning from exception

precise exceptions: no side effects from faulting instruction or after

- e.g. `pushq` that caused did not change `%rsp` before fault

- e.g. can't notice if instructions were executed in parallel

backup slides

swapping

early motivation for virtual memory: *swapping*

using disk (or SSD, ...) as the next level of the memory hierarchy
how our textbook and many other sources presents virtual memory

OS allocates *program space on disk*
own mapping of virtual addresses to location on disk

DRAM is a cache for disk

swapping

early motivation for virtual memory: *swapping*

using disk (or SSD, ...) as the next level of the memory hierarchy
how our textbook and many other sources presents virtual memory

OS allocates *program space on disk*
own mapping of virtual addresses to location on disk

DRAM is a cache for disk

swapping components

“swap in” a page — exactly like allocating on demand!

- OS gets page fault — invalid in page table

- check where page actually is (from virtual address)

- read from disk

- eventually restart process

“swap out” a page

- OS marks as invalid in the page table(s)

- copy to disk (if modified)

HDD/SDDs are slow

HDD reads and writes: milliseconds to tens of milliseconds

- minimum size: 512 bytes

- writing tens of kilobytes basically as fast as writing 512 bytes

SSD reads and writes: hundreds of microseconds

- designed for reads/writes of kilobytes (not much smaller)

HDD/SDDs are slow

HDD reads and writes: milliseconds to tens of milliseconds

- minimum size: 512 bytes

- writing tens of kilobytes basically as fast as writing 512 bytes

SSD reads and writes: hundreds of microseconds

- designed for reads/writes of kilobytes (not much smaller)

HDD/SDDs are slow

HDD reads and writes: milliseconds to tens of milliseconds

- minimum size: 512 bytes

- writing tens of kilobytes basically as fast as writing 512 bytes

SSD reads and writes: hundreds of microseconds

- designed for reads/writes of kilobytes (not much smaller)

HDD/SDDs are slow

HDD reads and writes: milliseconds to tens of milliseconds

- minimum size: 512 bytes

- writing tens of kilobytes basically as fast as writing 512 bytes

SSD reads and writes: hundreds of microseconds

- designed for reads/writes of kilobytes (not much smaller)

HDD/SDDs are slow

HDD reads and writes: milliseconds to tens of milliseconds

- minimum size: 512 bytes

- writing tens of *kilobytes* basically as fast as writing 512 bytes

SSD reads and writes: hundreds of microseconds

- designed for reads/writes of *kilobytes* (not much smaller)

HDD/SDDs are slow

HDD reads and writes: *milliseconds to tens of milliseconds*

- minimum size: 512 bytes

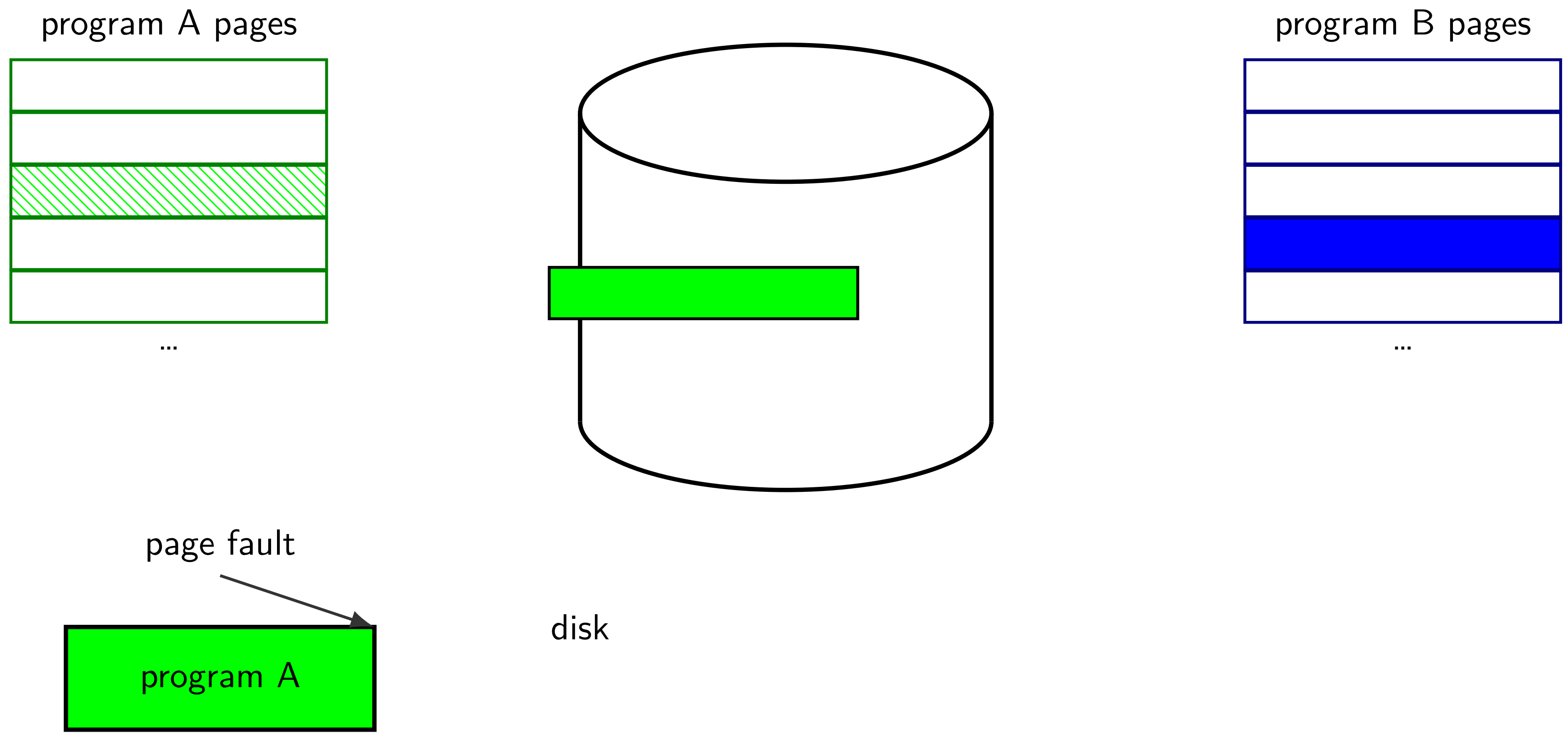
- writing tens of kilobytes basically as fast as writing 512 bytes

SSD reads and writes: *hundreds of microseconds*

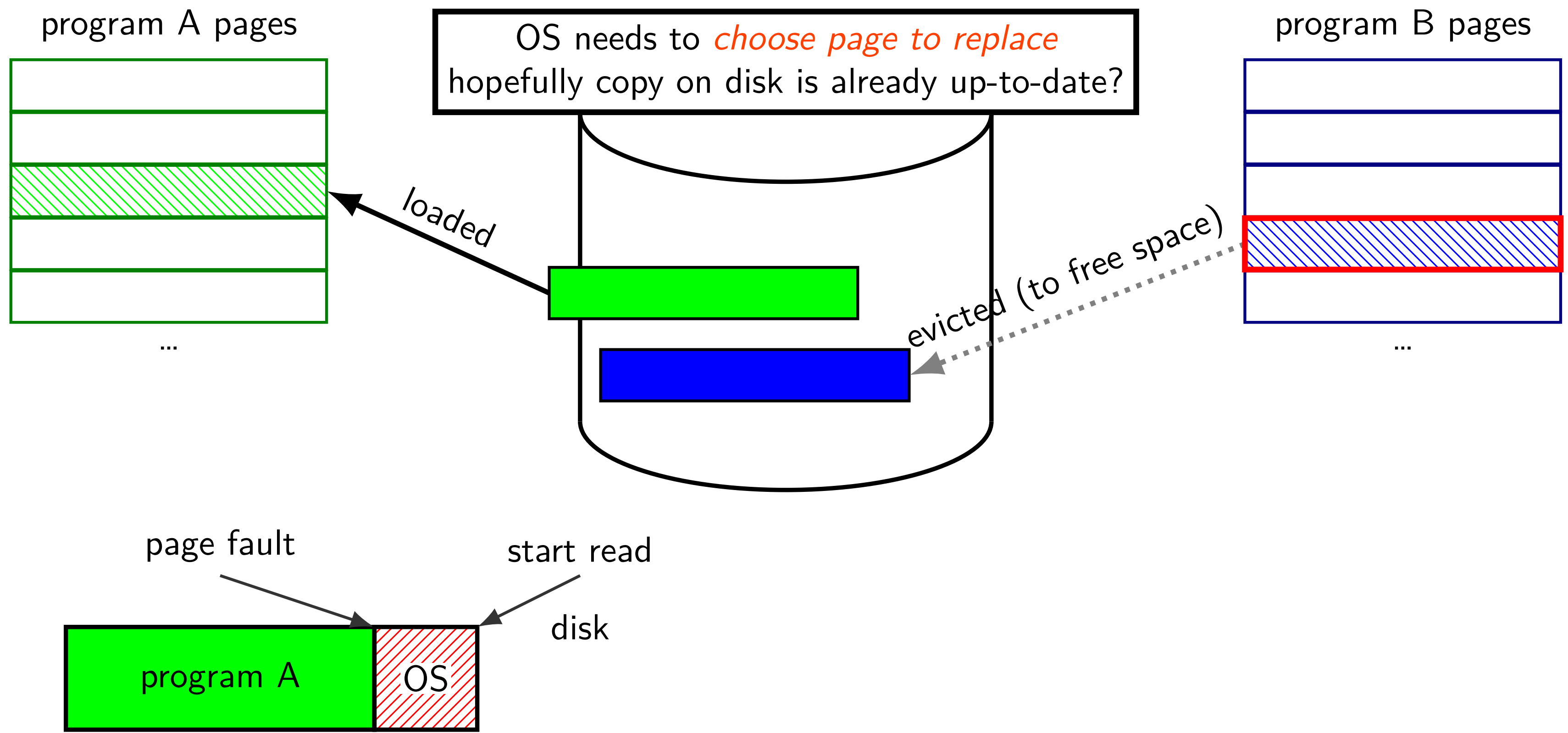
- designed for reads/writes of kilobytes (not much smaller)

swapping timeline

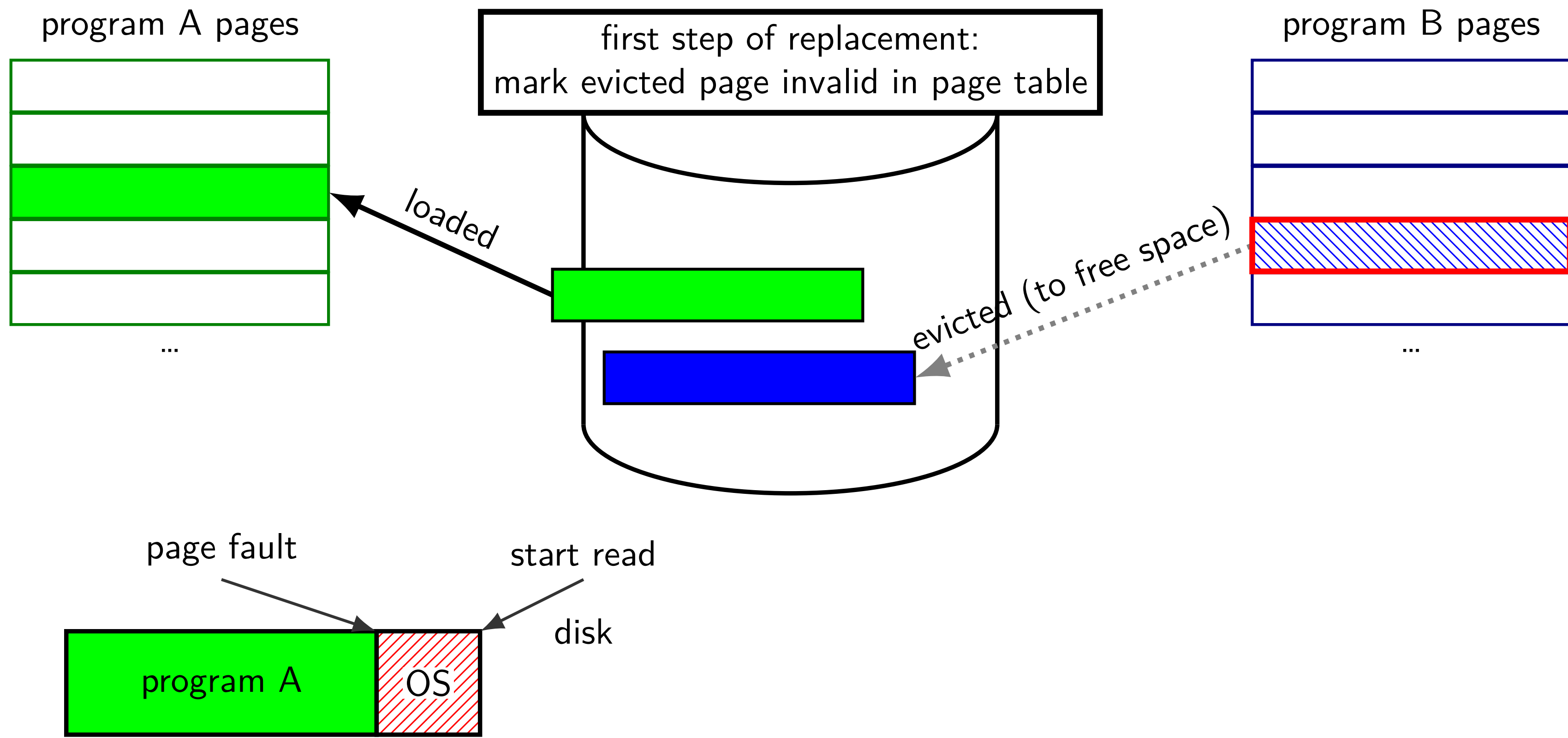
swapping timeline



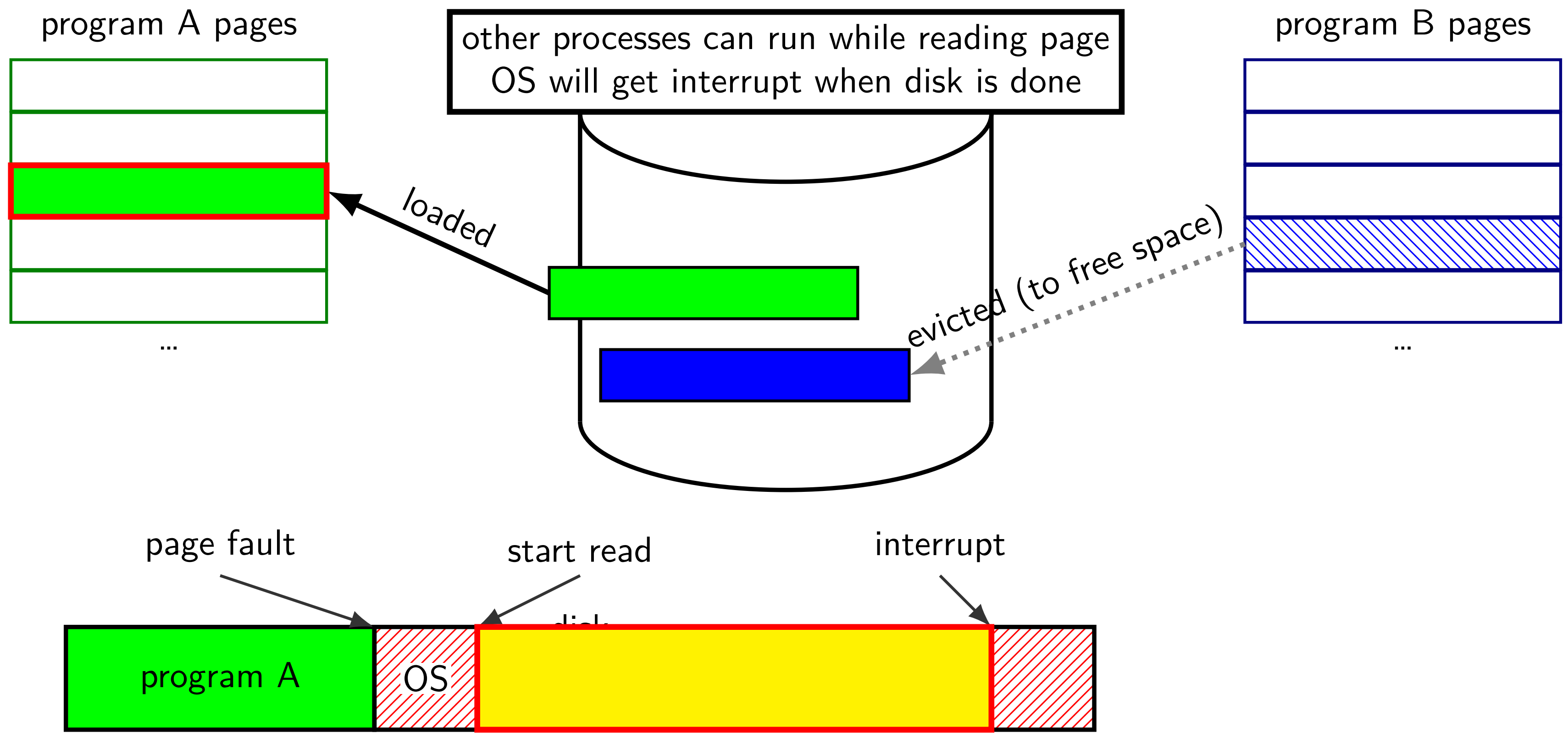
swapping timeline



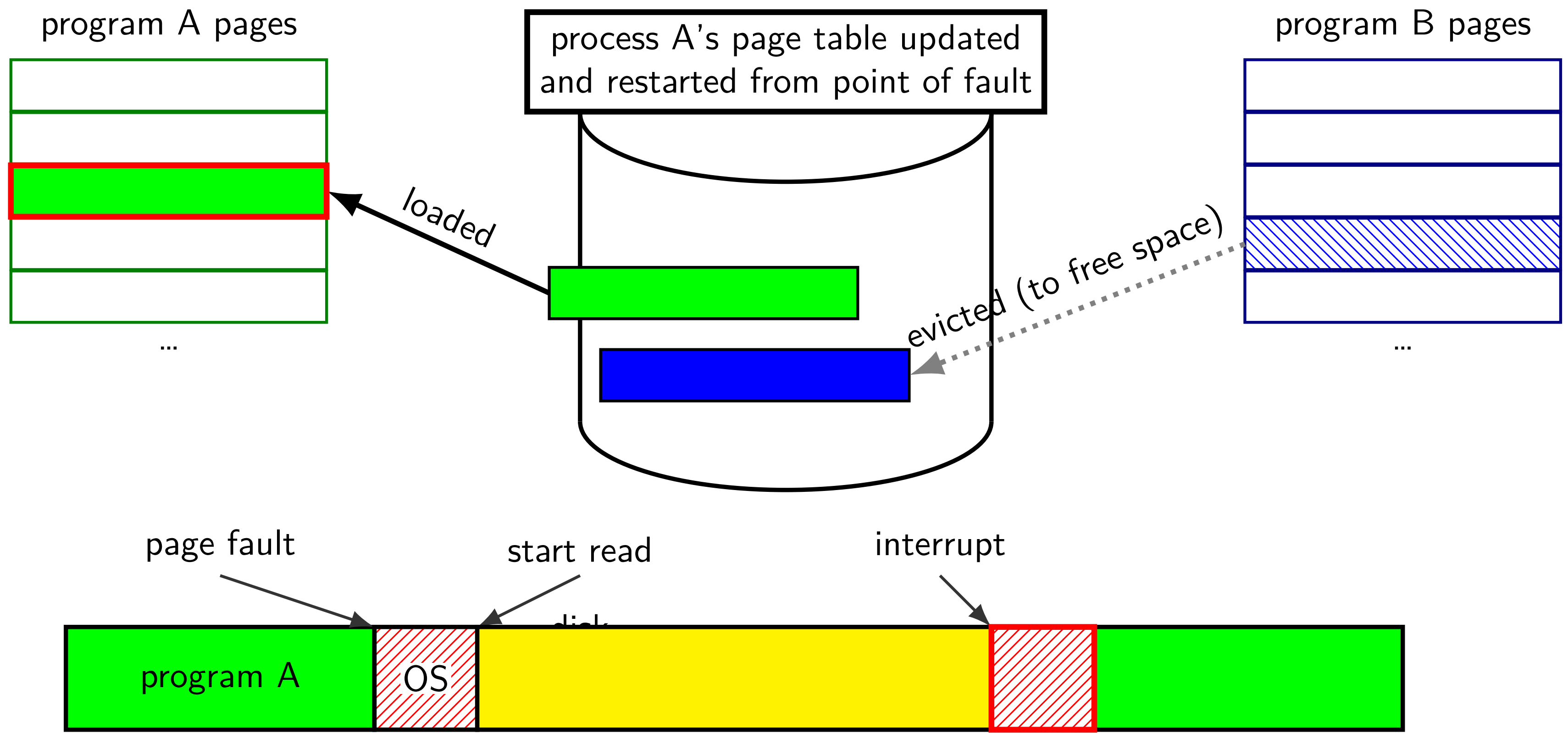
swapping timeline



swapping timeline



swapping timeline



swapping almost mmap

access mapped file for first time, read from disk
(like swapping when memory was swapped out)

write “mapped” memory, write to disk eventually
(like writeback policy in swapping)
use “dirty” bit

extra detail: other processes should see changes
all accesses to file use *same physical memory*

swap versus cache

swapping versus caching

“cache block”

approx

physical page

fully associative

every virtual page can be stored in any physical page

replacement/cache misses managed by the OS

normal cache hits happen in hardware

hardware's page table lookup

common case that needs to be very fast

copy-on-write and fork

fast copies

Unix mechanism for starting a new process: `fork()`

creates a *copy* of an entire program!

(usually, the copy then calls `execve` — replaces itself with another program)

how isn't this really slow?